

Day 39 : JSX, Babel, props and React Component

1. What is JSX Really?

JSX is **syntax extension for JavaScript**. It's not HTML, not a string, not a template language.

```
const element = <h1>Hello</h1>;
```

This is **neither HTML nor JavaScript**. It's JSX - a syntax that **looks like HTML** but is **JavaScript underneath**.

2. The Transformation Process

Step 1: You write JSX

```
<h1 id="title">Hello</h1>
```

Step 2: Babel transforms to JavaScript

```
React.createElement('h1', { id: 'title' }, 'Hello')
```

Step 3: React creates Virtual DOM object

```
{
  type: 'h1',
  props: {
    id: 'title',
    children: 'Hello'
  },
  key: null,
```

```
    ref: null  
  }
```

You already know step 3. JSX is just a way to **avoid writing** `createElement` manually.

3. Basic JSX Syntax

Single Element

```
const element = <h1>Hello</h1>;
```

With Attributes

```
const element = <h1 id="title" className="heading">Hello</h1>;
```

Self-Closing Tags

```
const element = ;  
const element = <input type="text" />;  
const element = <br />;
```

Rule: All tags must be closed. In HTML you can write `` but in JSX you **must** write ``.

4. Multi-line JSX

Use parentheses for readability:

```
const element = (  
  <div>  
    <h1>Title</h1>  
    <p>Paragraph</p>  
  </div>  
)
```

Without parentheses (works but risky):

```
const element = <div>
  <h1>Title</h1>
</div>;
```

Best practice: Always use parentheses for multi-line JSX.

5. Single Root Element Rule

JSX expression must return **ONE** element.

```
// ❌ Error - two root elements
const element = (
  <h1>Title</h1>
  <p>Text</p>
);

// ✅ Correct - wrapped in div
const element = (
  <div>
    <h1>Title</h1>
    <p>Text</p>
  </div>
);
```

Why? Because it transforms to:

```
// This doesn't make sense
React.createElement('h1', null, 'Title')
React.createElement('p', null, 'Text')
// Two separate function calls, can't assign to one variable

// This works
React.createElement('div', null,
  React.createElement('h1', null, 'Title'),
```

```
React.createElement('p', null, 'Text')
)
// One function call returning one object
```

6. JavaScript Expressions in JSX

Use `{ }` to embed JavaScript:

Variables

```
const name = "Rohit";
const element = <h1>Hello {name}</h1>;
// Output: <h1>Hello Rohit</h1>
```

Expressions

```
const a = 5;
const b = 10;
const element = <p>Sum: {a + b}</p>;
// Output: <p>Sum: 15</p>
```

Function Calls

```
function getName() {
  return "Rohit";
}

const element = <h1>Hello {getName()}</h1>;
```

Ternary Operator

```
const age = 20;
const element = <p>{age >= 18 ? "Adult" : "Minor"}</p>;
```

Logical AND

```
const isLoggedIn = true;  
const element = <div>{isLoggedIn && <p>Welcome back!</p>}</div>;  
// If isLoggedIn is true, shows <p>, otherwise nothing
```

7. What Can and Cannot Go Inside

Can Use

- Variables: `{name}`
- Expressions: `{5 + 10}`, `{a * b}`
- Function calls: `{getName()}`
- Ternary: `{x > 5 ? "big" : "small"}`
- Logical: `{isLoggedIn && <div>Show</div>}`
- Objects (for style): `{{color: 'red'}}`
- Arrays: `{[1, 2, 3].map(n => <p>{n}</p>)}`

Cannot Use

- Statements: `if`, `for`, `while`, `switch`
- Object directly: `{{name: "Rohit"}}` (shows nothing, not valid JSX child)

```
//  Wrong  
const element = (  
  <div>  
    {if (x > 5) { <p>Big</p> }}  
  </div>  
)  
);
```

```
//  Correct  
const element = (  
  <div>  
    {isLoggedIn && <p>Welcome back!</p>}
```

```

    <div>
      {x > 5 ? <p>Big</p> : <p>Small</p>}
    </div>
  );

// ✅ Also correct - use IIFE
const element = (
  <div>
    {(() => {
      if (x > 5) return <p>Big</p>;
      return <p>Small</p>;
    })()}
  </div>
);

```

8. Attributes in JSX

String Attributes

```

<h1 id="title">Hello</h1>


```

JavaScript Expressions as Attributes

```

const imageUrl = "photo.jpg";
<img src={imageUrl} />

const divId = "container";
<div id={divId}>Content</div>

```

Important: className not class

```
// ❌ Wrong
<div class="container">Content</div>

// ✅ Correct
<div className="container">Content</div>
```

Why? `class` is reserved keyword in JavaScript.

Important: htmlFor not for

```
// ❌ Wrong
<label for="name">Name</label>

// ✅ Correct
<label htmlFor="name">Name</label>
```

camelCase Event Handlers

```
// HTML
<button onclick="handleClick()">Click</button>

// JSX
<button onClick={handleClick}>Click</button>
```

Common ones:

- `onclick` → `onClick`
- `onchange` → `onChange`
- `onsubmit` → `onSubmit`

9. Style Attribute

In JSX, `style` is an **object**, not a string.

// ❌ Wrong (HTML way)

```
<h1 style="color: red; font-size: 20px;">Title</h1>
```

// ✅ Correct (JSX way)

```
<h1 style={{ color: 'red', fontSize: '20px' }}>Title</h1>
```

Why double braces?

- Outer `{ }` = JavaScript expression
- Inner `{ }` = JavaScript object

// Separated for clarity

```
const styleObj = { color: 'red', fontSize: '20px' };
```

```
<h1 style={styleObj}>Title</h1>
```

CSS properties become camelCase:

- `font-size` → `fontSize`
- `background-color` → `backgroundColor`
- `margin-top` → `marginTop`

```
const element = (  
  <div style={{  
    backgroundColor: 'blue',  
    fontSize: '16px',  
    padding: '10px',  
    marginTop: '20px'  
  }}>  
    Styled div  
  </div>  
)
```

10. Children in JSX

Text Children

```
<h1>Simple text</h1>
```

Expression Children

```
const name = "React";  
<h1>Learning {name}</h1>
```

Element Children

```
<div>  
  <h1>Title</h1>  
  <p>Text</p>  
</div>
```

No Children

```
  
<input type="text" />
```

Mixed Children

```
const count = 5;  
<div>  
  <h1>Count: {count}</h1>  
  <p>Some text</p>  
  <button>Click</button>  
</div>
```

11. Arrays in JSX

JSX can render arrays of elements:

```
const items = [  
  <li>Item 1</li>,  
  <li>Item 2</li>,  
  <li>Item 3</li>  
];  
  
const element = <ul>{items}</ul>;
```

Common pattern: map() to create elements

```
const numbers = [1, 2, 3, 4, 5];  
  
const element = (  
  <ul>  
    {numbers.map(num => (  
      <li>{num}</li>  
    ))}  
  </ul>  
);
```

Warning in console: "Each child should have a unique key prop"

```
// ✅ With key  
const element = (  
  <ul>  
    {numbers.map(num => (  
      <li key={num}>{num}</li>  
    ))}  
  </ul>  
);
```

We'll cover `key` in detail later. For now: **key helps React identify which items changed.**

12. Comments in JSX

```
const element = (  
  <div>  
    { /* This is a comment */ }  
    <h1>Title</h1>  
  
    { /*  
      Multi-line  
      comment  
    */ }  
    <p>Text</p>  
  </div>  
)
```

Note: Use `{ /* */ }` not `<!-- -->` (HTML comments don't work).

13. Fragments

Problem: Single root element creates unnecessary divs.

```
// Creates extra div in DOM  
const element = (  
  <div>  
    <h1>Title</h1>  
    <p>Text</p>  
  </div>  
)
```

Solution: **React Fragment**

```
// No extra element in DOM  
const element = (  
  <React.Fragment>  
    <h1>Title</h1>  
  </React.Fragment>  
)
```

```

    <p>Text</p>
  </React.Fragment>
);

// Shorthand
const element = (
  <>
    <h1>Title</h1>
    <p>Text</p>
  </>
);

```

Fragments let you group elements without adding extra DOM nodes.

14. Boolean, Null, Undefined in JSX

These values **render nothing**:

```

<div>{true}</div>    // Empty
<div>{false}</div>   // Empty
<div>{null}</div>    // Empty
<div>{undefined}</div> // Empty

```

Useful for conditional rendering:

```

const showMessage = true;

<div>
  {showMessage && <p>Message</p>}
  {/* If showMessage is false, nothing renders */}
</div>

```

Number 0 is rendered:

```

<div>{0}</div> // Shows: 0

```

Be careful:

```
const items = [];  
  
// ❌ Shows "0" if array is empty  
<div>{items.length && <p>Has items</p>}</div>  
  
// ✅ Correct  
<div>{items.length > 0 && <p>Has items</p>}</div>
```

15. JSX vs createElement - Side by Side

```
// JSX  
<h1 id="title">Hello</h1>  
  
// createElement  
React.createElement('h1', { id: 'title' }, 'Hello')
```

```
// JSX  
<div>  
  <h1>Title</h1>  
  <p>Text</p>  
</div>  
  
// createElement  
React.createElement('div', null,  
  React.createElement('h1', null, 'Title'),  
  React.createElement('p', null, 'Text')  
)
```

```
// JSX  
const name = "Rohit";  
<h1>Hello {name}</h1>
```

```
// createElement
const name = "Rohit";
React.createElement('h1', null, 'Hello ', name)
```

```
// JSX


// createElement
React.createElement('img', { src: 'photo.jpg' })
```

16. Key Takeaways

1. **JSX is syntax sugar** for `React.createElement()`
2. **One root element** required (or use Fragment)
3. `{ }` **for JavaScript** expressions (not statements)
4. **className not class, htmlFor not for**
5. **camelCase** attributes and event handlers
6. **style is object** with camelCase properties
7. **Close all tags** (self-closing for no children)
8. **Arrays** can be rendered directly
9. **Boolean/null/undefined** render nothing
10. **Babel transforms** JSX to JavaScript at runtime (with CDN)

React Component

1. What is a Component?

A **Component** is a JavaScript function that returns JSX.

```
// This is a Component
function Greeting() {
  return <h1>Hello World</h1>;
}
```

That's it. Simple.

2. Component vs Regular Function

Regular Function (what you already know)

```
function getGreeting() {
  return <h1>Hello World</h1>;
}
```

```
// Use it by calling
<div>{getGreeting()}</div>
```

Component (React way)

```
function Greeting() {
  return <h1>Hello World</h1>;
}
```

```
// Use it like an HTML tag
<div><Greeting /></div>
```

Key differences:

1. **Component name starts with Capital letter** (convention + requirement)
2. **Use JSX syntax** `<Greeting />` instead of calling `{Greeting()}`

3. React calls the function for you

3. Why Capital Letter?

React distinguishes between **DOM elements** and **Components** by the first letter:

```
// Lowercase = DOM element
<div />    // React creates: React.createElement('div', ...)
<h1 />    // React creates: React.createElement('h1', ...)

// Capital letter = Component
<Greeting /> // React creates: React.createElement(Greeting, ...)
<UserCard /> // React creates: React.createElement(UserCard, ...)
```

Under the hood:

```
// JSX
<div>Content</div>

// Becomes
React.createElement('div', null, 'Content')
// type is STRING 'div'

// JSX
<Greeting />

// Becomes
React.createElement(Greeting, null)
// type is FUNCTION Greeting
```

React sees:

- **String type** → create DOM element

- **Function type** → call function, render what it returns

4. Basic Component Example

```
function Welcome() {  
  return <h1>Welcome to React!</h1>;  
}
```

```
// Use it  
const app = (  
  <div>  
    <Welcome />  
    <Welcome />  
    <Welcome />  
  </div>  
);
```

```
// Output:  
// <div>  
//   <h1>Welcome to React!</h1>  
//   <h1>Welcome to React!</h1>  
//   <h1>Welcome to React!</h1>  
// </div>
```

Each `<Welcome />` tells React: "Call the Welcome function, render what it returns."

5. Components Can Return Complex JSX

```
function UserCard() {  
  return (  
    <div style={{ border: '1px solid black', padding: '10px' }}>  
      <h2>Rohit</h2>  
      <p>Age: 25</p>  
      <p>Location: India</p>  
    </div>  
  );  
}
```

```

    </div>
  );
}

const app = (
  <div>
    <UserCard />
  </div>
);

```

6. Components Inside Components (Composition)

```

function Header() {
  return <h1>My Website</h1>;
}

function Footer() {
  return <p>© 2024 My Website</p>;
}

function App() {
  return (
    <div>
      <Header />
      <p>This is the content</p>
      <Footer />
    </div>
  );
}

// Render App
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);

```

What happens:

1. React calls `App()`
2. App returns JSX with `<Header />` and `<Footer />`
3. React sees `<Header />` → calls `Header()`
4. React sees `<Footer />` → calls `Footer()`
5. All JSX combined and rendered

7. Props - Passing Data to Components

Problem: How to make components reusable with different data?

```
// This is hardcoded
function Greeting() {
  return <h1>Hello Rohit</h1>;
}
```

Solution: Use **props** (properties)

```
// Component receives props as parameter
function Greeting(props) {
  return <h1>Hello {props.name}</h1>;
}

// Pass data via attributes
const app = (
  <div>
    <Greeting name="Rohit" />
    <Greeting name="Alice" />
    <Greeting name="Bob" />
  </div>
);

// Output:
// <div>
```

```
// <h1>Hello Rohit</h1>
// <h1>Hello Alice</h1>
// <h1>Hello Bob</h1>
// </div>
```

8. What Are Props?

Props is an **object** containing all attributes you pass:

```
<Greeting name="Rohit" age={25} />
```

React creates:

```
const props = {
  name: "Rohit",
  age: 25
};
```

Then calls:

```
Greeting(props);
```

9. Accessing Props

```
function UserCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
      <p>Location: {props.location}</p>
    </div>
  );
}
```

```
// Use it
<UserCard name="Rohit" age={25} location="India" />
```

What React does:

```
// React creates props object
const props = {
  name: "Rohit",
  age: 25,
  location: "India"
};

// React calls
UserCard(props);
```

10. Destructuring Props (Cleaner Syntax)

Instead of writing `props.name` , `props.age` everywhere:

```
// Without destructuring
function UserCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>Age: {props.age}</p>
    </div>
  );
}

// With destructuring (preferred)
function UserCard({ name, age }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
    </div>
  );
}
```

```
    </div>  
  );  
}
```

Both are same, destructuring is just cleaner.

11. Different Types of Props

String Props

```
<Greeting name="Rohit" />  
  
function Greeting({ name }) {  
  return <h1>Hello {name}</h1>;  
}
```

Number Props

```
<UserCard age={25} />  
  
function UserCard({ age }) {  
  return <p>Age: {age}</p>;  
}
```

Boolean Props

```
<Message isImportant={true} />  
  
function Message({ isImportant }) {  
  return <p>{isImportant ? "IMPORTANT!" : "Regular message"}</p>;  
}
```

Array Props

```
<SkillsList skills={['React', 'JavaScript', 'CSS']} />
```

```
function SkillsList({ skills }) {  
  return (  
    <ul>  
      {skills.map((skill, index) => (  
        <li key={index}>{skill}</li>  
      ))}  
    </ul>  
  );  
}
```

Object Props

```
<UserCard user={{ name: "Rohit", age: 25 }} />
```

```
function UserCard({ user }) {  
  return (  
    <div>  
      <h2>{user.name}</h2>  
      <p>Age: {user.age}</p>  
    </div>  
  );  
}
```

Function Props

```
<Button onClick={() => alert("Clicked!")} />
```

```
function Button({ onClick }) {  
  return <button onClick={onClick}>Click Me</button>;  
}
```

12. Props Are Read-Only

You cannot modify props:

```
function Greeting({ name }) {  
  // ❌ Wrong - props are immutable  
  name = "Something else";  
  
  return <h1>Hello {name}</h1>;  
}
```

Props are **read-only**. If you want to change data, you'll use **state** (next topic after this).

13. Default Props

What if prop is not provided?

```
function Greeting({ name }) {  
  return <h1>Hello {name}</h1>;  
}  
  
<Greeting /> // name is undefined  
// Output: Hello undefined
```

Solution 1: Default parameter

```
function Greeting({ name = "Guest" }) {  
  return <h1>Hello {name}</h1>;  
}  
  
<Greeting /> // Output: Hello Guest  
<Greeting name="Rohit" /> // Output: Hello Rohit
```

Solution 2: Conditional


```
function Greeting({ name }) {  
  return <h1>Hello {name || "Guest"}</h1>;  
}
```

14. Children Prop (Special Prop)

Content between opening and closing tags becomes `props.children` :

```
function Card({ children }) {  
  return (  
    <div style={{ border: '1px solid black', padding: '10px' }}>  
      {children}  
    </div>  
  );  
}  
  
// Use it  
<Card>  
  <h2>Title</h2>  
  <p>This is content</p>  
</Card>  
  
// children prop receives:  
// <h2>Title</h2>  
// <p>This is content</p>
```

Another example:

```
function Button({ children }) {  
  return <button>{children}</button>;  
}  
  
<Button>Click Me</Button>  
// children = "Click Me"
```

```
<Button>
  <span>👍</span> Like
</Button>
// children = [<span>👍</span>, " Like"]
```

15. Component Naming Rules

✓ Valid Names

```
function MyComponent() {}
function Header() {}
function UserCard() {}
function App() {}
function Button123() {}
```

✗ Invalid Names

```
function myComponent() {} // Must start with capital
function my-component() {} // No hyphens
function 123Button() {} // Can't start with number
```

Convention:

- **PascalCase** (capital first letter, no spaces/hyphens)
- Descriptive names: `UserCard` not `UC`

16. When to Create a Component?

Create a component when:

1. **Code is reused** - same UI in multiple places
2. **UI is complex** - break into smaller pieces
3. **Logical separation** - Header, Footer, Sidebar

```
// Bad - everything in one component
function App() {
  return (
    <div>
      <header>
        <h1>Title</h1>
        <nav>...</nav>
      </header>
      <main>
        <article>...</article>
        <aside>...</aside>
      </main>
      <footer>...</footer>
    </div>
  );
}

// Good - split into components
function Header() { return <header>...</header>; }
function Main() { return <main>...</main>; }
function Footer() { return <footer>...</footer>; }

function App() {
  return (
    <div>
      <Header />
      <Main />
      <Footer />
    </div>
  );
}
```

17. Key Takeaways

1. **Component = function returning JSX**
 2. **Capital letter required** - distinguishes from DOM elements
 3. **Use like HTML tags** - `<Component />`
 4. **Props = data passed to component** - `<Component name="value" />`
 5. **Props is an object** - `{ name: "value" }`
 6. **Props are read-only** - cannot modify
 7. **Destructure props** for cleaner code
 8. **children prop** - content between tags
 9. **Compose components** - components inside components
 10. **Break UI into components** - reusability and maintainability
-